

Reinforcement Learning - Part 2

Deep RL, DQN, Policy Methods, MDP, Bellman, Advanced Topics

GARP RAI Exam Preparation

What You Will Learn in Part 2:

Topic	Key Concepts Covered
5. Deep RL & DQN	Neural Networks, Experience Replay, Target Networks
6. Policy Methods	Value-based vs Policy-based, Actor-Critic, Continuous Actions
7. MDP & Bellman	Markov Property, Dynamic Programming, Optimality Equations
8. Advanced Topics	RLHF, AlphaGo, Model-Based RL, Continuous RL

Key to Success: Understand the "exploration vs exploitation" trade-off!

Deep Reinforcement Learning - Scaling RL

The Problem with Tabular Methods:

- Q-table stores $Q(s,a)$ for every state-action pair
- Number of states can be **enormous** (or infinite)
- Example: Atari games have 10^{50} possible states!
- Self-driving cars have continuous state spaces

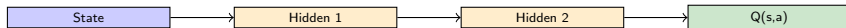
The Solution: Deep RL

- Use **neural networks** to approximate $Q(s,a)$
- $Q(s, a) \approx Q(s, a; \theta)$ where θ = network weights
- Neural network takes state as input, outputs Q-values for all actions

Why Neural Networks?

- 1 **Generalization**: Similar states get similar Q-values
- 2 **Compact representation**: No need to store all states
- 3 **Continuous spaces**: Can handle continuous state inputs

Deep RL Architecture:



Exam Trap: Deep RL is needed when state spaces are too large for tables!

DQN - Introduction

Deep Q-Network - The Breakthrough

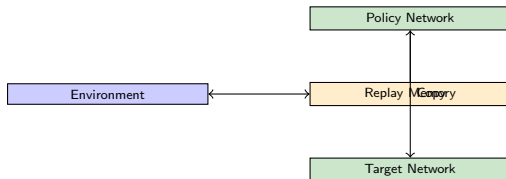
What is DQN?

- Combines Q-learning with [deep neural networks](#)
- Developed by DeepMind (2015)
- Achieved human-level performance on Atari games

Three Key Innovations:

- 1 [Experience Replay](#): Store experiences and sample randomly
- 2 [Target Network](#): Separate network for stable targets
- 3 [\$\epsilon\$ -Greedy](#): For exploration

DQN Architecture:



How DQN Works:

- 1 Interact with environment, store experiences
- 2 Sample random mini-batch from replay memory
- 3 Update policy network using TD target from target network
- 4 Periodically copy policy network weights to target network

Exam Trap: DQN's success comes from experience replay and target network!

DQN - Experience Replay

Experience Replay - Why It's Critical

The Problem:

- Consecutive experiences in RL are **highly correlated**
- Example: In a game, consecutive frames are nearly identical
- Correlated data breaks the i.i.d. assumption of supervised learning

The Solution: Experience Replay

- 1 Store experiences (s, a, r, s') in a **replay memory**
- 2 During training, sample **random mini-batches** from memory
- 3 This breaks temporal correlation

Why Random Sampling?

- Reduces correlation between samples
- Makes learning more stable
- Reuses experiences multiple times (more efficient)

Replay Memory Structure:

- Fixed size (e.g., 1,000,000 experiences)
- Oldest experiences are discarded (FIFO)
- Random sampling (uniform or prioritized)

Benefits:

- 1 **Stable learning**: Breaks correlations
- 2 **Efficient**: Reuses experiences
- 3 **Reduces variance**: Averaging over random samples

Prioritized Experience Replay:

- Sample experiences with **higher TD error** more often
- Focuses on experiences that are most "surprising"

Exam Trap: Experience replay breaks temporal correlation!

DQN - Target Network

Target Network - Stabilizing the Moving Target

The Problem:

- Q-learning uses $R + \gamma \max_{a'} Q(s', a')$ as target
- But $Q(s', a')$ is the **same network** being updated!
- This creates a "**chasing the moving target**" problem

The Solution: Target Network

- Use a **separate network** for the target
- Target network weights: θ^-
- Policy network weights: θ

How It Works:

- 1 Policy network: Used to select actions (ϵ -greedy)
- 2 Target network: Used to compute TD target
- 3 Update policy network: $L = (Q(s, a; \theta) - (R + \gamma \max_{a'} Q(s', a'; \theta^-)))^2$
- 4 Periodically copy: $\theta^- \leftarrow \theta$

Update Frequency:

- Typical: Copy every **500** steps
- Too frequent: Target moves too fast (unstable)
- Too infrequent: Target is outdated (slow learning)

Benefits:

- 1 **Stable targets**: Target doesn't change every step
- 2 **More stable learning**: Reduces oscillations
- 3 **Convergence**: Helps converge to optimal policy

Exam Trap: Target network is updated periodically!

DQN Loss Function and Training

Loss Function (MSE):

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} [(Q(s, a; \theta) - (r + \gamma \max_{a'} Q(s', a'; \theta^-)))^2]$$

Components:

- $Q(s, a; \theta)$: Policy network prediction
- $r + \gamma \max_{a'} Q(s', a'; \theta^-)$: TD target from target network
- $U(D)$: Uniform random sampling from replay memory

Training Process:

- 1 Sample random mini-batch from replay memory
- 2 For each experience:
 - 1 Compute target: $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$
 - 2 Compute prediction: $Q(s, a; \theta)$
 - 3 Compute loss: $(y - Q(s, a; \theta))^2$
- 3 Average loss over mini-batch
- 4 Backpropagate and update θ

Gradient Update:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$$

Hyperparameters:

- Learning rate α : 0.001 (typical)
- Mini-batch size: 32 or 64
- Replay memory size: 1,000,000

Exam Trap: DQN loss is MSE between policy and target networks!

DQN - Training Algorithm

DQN Training - Step-by-Step Process

Algorithm:

1 Initialize:

- Policy network $Q(s, a; \theta)$ with random weights
- Target network $Q(s, a; \theta^-)$ with same weights
- Replay memory D of capacity N

2 For each episode:

1 Initialize state s

2 For each step:

- 1 With probability ϵ : Choose random action (explore)
- 2 Otherwise: Choose $a = \arg \max_a Q(s, a; \theta)$ (exploit)
- 3 Take action a , observe reward r and next state s'
- 4 Store (s, a, r, s') in D
- 5 Sample random mini-batch from D
- 6 Compute loss and update θ
- 7 Periodically update $\theta^- \leftarrow \theta$
- 8 Set $s \leftarrow s'$

ϵ Decay in DQN:

- Start with $\epsilon = 1.0$ (pure exploration)
- Decay to $\epsilon = 0.01$ (mostly exploitation)
- Linear decay over first 1,000,000 steps

Stopping Criteria:

- Maximum number of steps reached
- No improvement in Q-values
- Episode reward converges

Exam Trap: DQN has two networks: policy and target!

DQN - Convergence and Stability

DQN - Ensuring Stable Learning

Challenges in Deep RL:

- 1 **Non-stationarity**: The target distribution changes
- 2 **Correlation**: Consecutive samples are correlated
- 3 **Divergence**: Q-learning with function approximation can diverge

DQN Solutions:

- 1 **Experience Replay**: Breaks correlation
- 2 **Target Network**: Stabilizes targets
- 3 **Gradient Clipping**: Prevents exploding gradients
- 4 **Frame Stacking**: Gives temporal context

Gradient Clipping:

- Clip gradients to a maximum value
- Prevents very large updates
- Common threshold: 1.0 or 10.0

Frame Stacking (for Atari):

- Stack last 4 frames
- Provides temporal information (motion)
- Input: 84x84x4 image

Double DQN:

- Uses target network for action selection too
- Reduces overestimation bias
- $y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$

Dueling DQN:

- Separate value and advantage streams
- Better estimate of state values
- $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$

Exam Trap: DQN can be unstable - use experience replay and target networks!

DQN Extensions - Making It Better

Double DQN:

- Uses target network for action selection too
- Reduces overestimation bias
- $y = r + \gamma Q(s', \arg \max_a Q(s', a'; \theta^-); \theta^-)$

Dueling DQN:

- Separate value and advantage streams
- Better estimate of state values
- $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_a A(s, a'))$

Prioritized Experience Replay:

- Sample experiences with higher TD error more often
- Focuses on most "surprising" experiences
- Can improve learning efficiency

Rainbow:

- Combines multiple DQN improvements:
 - Double DQN
 - Dueling DQN
 - Prioritized Experience Replay
 - Multi-step learning
 - Distributional RL
 - Noisy Nets
- State-of-the-art performance

Exam Trap: Double DQN and Dueling DQN are key extensions!

DQN - Advantages and Limitations

DQN - Pros, Cons, and When to Use

Advantages:

- 1 Handles large state spaces - neural networks generalize
- 2 State-of-the-art on many games
- 3 Can learn directly from raw pixels
- 4 Combines deep learning with RL

Limitations:

- 1 Computationally expensive - training can take days
- 2 Requires many samples - millions of interactions
- 3 Limited to discrete action spaces - outputs Q-values per action
- 4 Can be unstable - hyperparameter sensitive

When to Use DQN:

- Large discrete action spaces
- High-dimensional state spaces (images)
- When you have computational resources
- When samples are plentiful

When NOT to Use DQN:

- Small state spaces (tabular methods are better)
- Continuous action spaces (use policy-based methods)
- When samples are expensive

Exam Trap: DQN is only for discrete action spaces!

DQN - Complete Summary

Key Concepts:

1 DQN = Q-Learning + Deep Neural Networks

- Replaces Q-table with neural network
- Input: state; Output: Q-values for all actions

2 Three Key Innovations:

- 1 **Experience Replay**: Random sampling breaks correlation
- 2 **Target Network**: Separate network for stable targets
- 3 **ϵ -Greedy**: Exploration/exploitation balance

3 Loss Function:

$$L = \mathbb{E}[(Q(s, a; \theta) - (R + \gamma \max_{a'} Q(s', a'; \theta^-)))^2]$$

4 Update Steps:

- 1 Sample mini-batch from replay memory
- 2 Compute targets using target network
- 3 Update policy network
- 4 Periodically copy weights to target network

5 Limitations:

- Discrete actions only
- Computationally expensive
- Sample inefficient

Exam Success: Remember DQN's three key innovations!

Policy-Based Methods - Learning Strategy Directly

Value-Based vs Policy-Based:

Value-Based	Policy-Based
Learn $Q(s,a)$	Learn $\pi(a s)$ directly
Derive policy from Q	Policy is the output
Q-learning, SARSA	Policy Gradient
Discrete actions only	Continuous actions possible

Policy Representation:

- Policy parameterized: $\pi_{\theta}(a|s)$
- θ = parameters (neural network weights)
- Neural network takes state s as input
- Output: probability distribution over actions

Policy Gradient:

- Optimize θ to maximize expected reward
- Gradient ascent on $J(\theta) = E_{\pi}[\sum R]$
- Update: $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

Why Policy Methods?

- 1 Continuous action spaces: Can output continuous actions
- 2 Stochastic policies: Natural for partial observability
- 3 Better convergence: Guaranteed to converge to local optimum

Exam Trap: Policy methods are preferred for continuous action spaces!

REINFORCE Algorithm

REINFORCE - Monte Carlo Policy Gradient

The REINFORCE Algorithm:

- Monte Carlo policy gradient method
- Updates after complete episodes
- Uses the actual return G_t

Update Formula:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Intuition:

- If G_t is positive (good outcome), increase probability of action
- If G_t is negative (bad outcome), decrease probability of action
- The gradient $\nabla_{\theta} \log \pi_{\theta}$ points in the direction to increase the action's probability

Algorithm Steps:

- 1 Initialize policy parameters θ
- 2 For each episode:
 - 1 Generate episode: $s_0, a_0, r_1, s_1, \dots, s_T$
 - 2 For each step t :
 - 1 Compute $G_t = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$
 - 2 Update $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$

Advantages:

- Simple to implement
- Theoretically sound
- Works for continuous actions

Disadvantages:

- High variance (like MC)
- Requires complete episodes
- Slow convergence

Exam Trap: REINFORCE is MC-based with high variance!

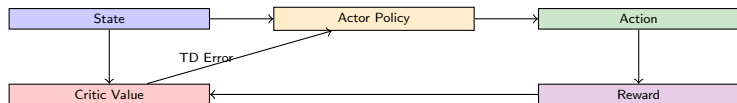
Actor-Critic Methods

Actor-Critic - The Best of Both Worlds

What is Actor-Critic?

- Combines **value-based** and **policy-based** methods
- **Actor**: Learns the policy (policy-based)
- **Critic**: Learns the value function (value-based)
- Critic evaluates actor's actions

Architecture:



How It Works:

- 1 Actor takes action based on current policy
- 2 Critic evaluates the action (TD error)
- 3 TD error: $\delta = R + \gamma V(s') - V(s)$
- 4 If $\delta > 0$: Action was better than expected $\rightarrow$ increase probability
- 5 If $\delta < 0$: Action was worse than expected $\rightarrow$ decrease probability

Advantages:

- 1 Lower **variance** than policy gradient
- 2 Can **learn online** (TD-based)
- 3 **Continuous action spaces** (policy-based)

Exam Trap: Actor-Critic combines policy (actor) and value (critic)!

Policy Methods - Summary

Policy Methods - Complete Summary

Key Concepts:

1 Policy-Based Methods:

- Learn policy $\pi_{\theta}(a|s)$ directly
- Output: action (or probability distribution)
- Better for continuous action spaces

2 Value-Based Methods:

- Learn $Q(s,a)$ or $V(s)$
- Output: value estimates
- Derive policy from values
- Better for discrete action spaces

3 REINFORCE:

- MC policy gradient
- High variance
- Requires complete episodes

4 Actor-Critic:

- Actor: Policy
- Critic: Value function
- Lower variance
- Can learn online

5 Advantages of Policy Methods:

- Continuous actions
- Stochastic policies
- Better convergence

6 Disadvantages of Policy Methods:

- Slow convergence
- Local optima
- Sample inefficient

Exam Success: Choose between value-based and policy-based based on action space!

MDP - Formal Definition

Markov Decision Processes - The RL Framework

What is an MDP?

- Mathematical framework for modeling decision-making
- Defined by: States, Actions, Transition Probabilities, Rewards
- Assumes the **Markov property**

MDP Components:

- 1 **States (S)**: Set of all possible states
- 2 **Actions (A)**: Set of all possible actions
- 3 **Transition Probability**: $P(s'|s, a)$
- 4 **Reward**: $R(s, a)$ or $R(s, a, s')$
- 5 **Discount Factor**: $\gamma \in [0, 1]$

The Markov Property:

$$P(S_{t+1}|S_t, S_{t-1}, \dots, S_1) = P(S_{t+1}|S_t)$$

- Future depends only on the **current state**
- "Memoryless" property
- Simplifies analysis significantly

Transition Probability Matrix:

- P = matrix where $P_{ij} = P(S_{t+1} = j | S_t = i)$
- For actions: $P_{ij}^a = P(S_{t+1} = j | S_t = i, A_t = a)$
- After n steps: P^n (matrix multiplication)

Example - Simple 2-State MDP:

- States: S_1, S_2
- Actions: Left, Right
- Transition: $P(\text{Left from } S_1 \text{ to } S_2) = 0.7, P(\text{Left from } S_1 \text{ to } S_1) = 0.3$

Exam Trap: MDPs assume the Markov property!

Bellman Equations - Derivation

Bellman Equations - The Recursive Foundation

The Bellman Equation (For a Fixed Policy):

$$V(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$$

Derivation:

- ① $V(s) = E[G_t | S_t = s]$
- ② $G_t = R_{t+1} + \gamma G_{t+1}$
- ③ $V(s) = E[R_{t+1} + \gamma V(S_{t+1}) | S_t = s]$
- ④ Expand expectation over actions and next states

Bellman Optimality Equation:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^*(s')]$$

For Q-Function:

$$Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a) + \gamma \max_{a'} Q(s', a')]$$

What They Tell Us:

- Value of a state = immediate reward + discounted value of next state
- This is [recursive](#)
- Can be solved using [dynamic programming](#)

Solving Bellman Equations:

- If transition probabilities known: Dynamic Programming
- If not known: MC or TD methods

Exam Trap: Bellman equations are recursive!

Bellman Equations - Worked Example

Bellman Equations - Worked Example

Example - Simple MDP:

- 2 states: S_1, S_2
- 2 actions: A_1, A_2 (deterministic)
- Rewards: $R(S_1, A_1, S_2) = 10, R(S_2, A_2, S_1) = 5$
- $\gamma = 0.9$

Bellman Equations:

- $V(S_1) = 10 + \gamma V(S_2)$
- $V(S_2) = 5 + \gamma V(S_1)$

Solve:

- $V(S_1) = 10 + 0.9V(S_2)$
- $V(S_2) = 5 + 0.9V(S_1)$
- Substitute: $V(S_1) = 10 + 0.9(5 + 0.9V(S_1)) = 10 + 4.5 + 0.81V(S_1)$
- $0.19V(S_1) = 14.5 \Rightarrow V(S_1) = 76.32$
- $V(S_2) = 5 + 0.9(76.32) = 73.69$

Value Iteration:

- Initialize $V(s) = 0$ for all s
- Iteratively apply Bellman optimality
- Converges to V^*

Policy Iteration:

- Initialize policy randomly
- Evaluate policy: compute $V(s)$
- Improve policy: $\pi'(s) = \arg \max_a Q(s, a)$
- Repeat until convergence

Exam Trap: Bellman equations can be solved analytically for simple MDPs!

Dynamic Programming in RL

Dynamic Programming - Solving MDPs

When DP Can Be Used:

- Environment model is known (transition probabilities)
- Reward function is known
- Finite state and action spaces

Policy Evaluation:

- Compute $V(s)$ for a given policy π
- Iterative update: $V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a)[R(s, a) + \gamma V_k(s')]$
- Converges to V^π

Policy Improvement:

- Given $V(s)$, find better policy
- $\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a)[R(s, a) + \gamma V(s')]$
- If $\pi' \neq \pi$, repeat

Policy Iteration:

- 1 Initialize policy π
- 2 Policy Evaluation: Compute V^π
- 3 Policy Improvement: Compute π' from V^π
- 4 If $\pi' \neq \pi$, go to step 2

Value Iteration:

- 1 Initialize $V(s) = 0$
- 2 Update: $V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a)[R(s, a) + \gamma V_k(s')]$
- 3 Converges to V^*

Exam Trap: DP requires complete knowledge of the environment!

RLHF - Reinforcement Learning from Human Feedback

RLHF - The Secret Behind Modern LLMs

What is RLHF?

- Uses **human feedback** as a reward signal
- Humans rank or score model outputs
- RL algorithm adjusts to **maximize human scores**

Why It Matters:

- Key to ChatGPT and other LLMs
- Improves **instruction following**
- Enforces **policy constraints**
- Enhances **reasoning coherence**

How It Works:

- 1 Train a reward model on human preferences
- 2 Fine-tune policy using RL (PPO)
- 3 Generate responses and get feedback
- 4 Update policy to maximize reward

RLHF Process:

- 1 **Step 1:** Collect human preference data
 - Show model outputs to humans
 - Humans rank or score them
- 2 **Step 2:** Train reward model
 - Supervised learning on human preferences
 - Reward model predicts human scores
- 3 **Step 3:** Fine-tune policy with RL
 - Use reward model as reward signal
 - Policy generates responses, gets reward
 - Update policy to maximize reward

Exam Trap: RLHF is for LLM fine-tuning!

Model-Based vs Model-Free RL

Model-Based vs Model-Free RL

Model-Based RL:

- Uses a **model** of the environment
- Can plan ahead using the model
- More sample efficient
- Risk: **Model bias**
- Example: AlphaZero

Model-Free RL:

- Learns directly from experience
- No environment model needed
- Less sample efficient
- No model bias
- Example: DQN, PPO

Comparison:

Model-Based	Model-Free
Uses environment model	No model needed
More sample efficient	Less sample efficient
Risk of model bias	No model bias
Planning possible	No planning
Works in known environments	Works in unknown environments

Examples:

- **AlphaZero**: Model-based (uses game rules)
- **DQN**: Model-free (learns from pixels)
- **Self-driving cars**: Often model-based (use simulation)

Exam Trap: Model-based = more efficient but risk of bias!

AlphaGo and AlphaZero

AlphaGo/AlphaZero - RL Success Stories

AlphaGo (2016):

- First computer to beat world champion in Go
- Used [Monte Carlo Tree Search](#) + Deep RL
- Trained on human games + self-play

AlphaGo Zero (2017):

- Learned only from self-play
- No human data used
- Surpassed AlphaGo in 3 days

AlphaZero (2018):

- Same algorithm for Chess, Shogi, Go
- Superhuman in all three games
- Model-based RL (uses game rules)

Key Innovations:

- 1 [Monte Carlo Tree Search \(MCTS\)](#): Planning
- 2 [Self-play](#): Generates training data
- 3 [Neural Networks](#): Policy and value networks
- 4 [Model-based](#): Uses game rules

Why Important:

- Demonstrated RL's power
- Discovered novel strategies
- Changed how AI is developed

Exam Trap: AlphaZero is model-based RL!

Continuous RL - SAC and PPO

Continuous Action Spaces - SAC and PPO

Why Continuous Actions Matter:

- Robotics: Motor torques, joint angles
- Control systems: Temperature, pressure
- Finance: Portfolio weights

SAC (Soft Actor-Critic):

- State-of-the-art for continuous actions
- Entropy regularization for exploration
- Off-policy (sample efficient)
- Uses two Q-networks (clipped double Q)

SAC Key Features:

- 1 **Maximum entropy RL:** Balances reward and exploration
- 2 **Off-policy:** Reuses old experiences
- 3 **Continuous actions:** Natural for robotics

PPO (Proximal Policy Optimization):

- Stable and reliable
- Uses clipped objective
- On-policy (less sample efficient)
- Most widely used in industry

PPO Key Features:

- 1 **Clipped surrogate objective:** Prevents large updates
- 2 **Multiple epochs:** Reuses data efficiently
- 3 **Stable training:** Reliable performance

Exam Trap: SAC is off-policy; PPO is on-policy!

RL - Complete Summary

Reinforcement Learning - Complete Summary

Core Concepts:

- Agent learns through **trial and error**
- Goal: **Maximize cumulative reward**
- Policy: Strategy for choosing actions
- Value functions: $V(s)$ and $Q(s,a)$

The Three Strategies (MAB):

- 1 Greedy (Pure Exploitation)
- 2 Random (Pure Exploration)
- 3 ϵ -Greedy (Balanced) - **Best practical choice**

Two Main Approaches:

1 Monte Carlo (MC):

- Episode-end updates
- Uses actual returns G_t
- Unbiased, high variance
- Episodic tasks only

2 Temporal Difference (TD):

- Step-by-step updates
- Uses bootstrapping
- Biased, low variance
- Episodic and continuous tasks

Two Families of Algorithms:

1 Value-Based:

- Q-learning (off-policy)
- SARSA (on-policy)
- DQN (deep learning)

2 Policy-Based:

- REINFORCE (MC)
- Actor-Critic
- Policy Gradient

Reinforcement Learning - Comparison Table

Method	Pros	Cons
MAB	Simple, fast	Single state only
MC	Unbiased	High variance, episodic
TD	Low variance, online	Biased
Q-Learning	Off-policy, simple	Discrete actions
SARSA	On-policy, safe	Conservative
DQN	Large state spaces	Discrete actions only
Policy Gradient	Continuous actions	High variance
Actor-Critic	Best of both	Complex

When to Use Each:

Situation	Recommended Method
Small state space	Tabular Q-learning or SARSA
Large state space	DQN or Actor-Critic
Continuous actions	Policy Gradient or Actor-Critic
Safety critical	SARSA (conservative)
Optimal policy	Q-learning (aggressive)

Exam Trap: Choose the right method for the problem!

Top 15 RL Exam Traps - Part 2

- 1 **Trap 1:** DQN uses **experience replay** + **target network** for stability
 - 2 **Trap 2:** Policy-based methods handle **continuous action spaces**
 - 3 **Trap 3:** Value-based methods handle **discrete action spaces**
 - 4 **Trap 4:** Actor-Critic = **Actor** (policy) + **Critic** (value)
 - 5 **Trap 5:** REINFORCE is **MC-based** with high variance
 - 6 **Trap 6:** MDPs assume the **Markov property**
 - 7 **Trap 7:** Bellman equations are **recursive**
 - 8 **Trap 8:** Model-based RL = **more efficient** but risk of model bias
 - 9 **Trap 9:** Model-free RL = **less efficient** but no model bias
 - 10 **Trap 10:** RLHF is **RL from human feedback**
 - 11 **Trap 11:** RLHF is used for **LLM fine-tuning**
 - 12 **Trap 12:** AlphaZero is **model-based** RL
 - 13 **Trap 13:** SAC is **off-policy** (continuous actions)
 - 14 **Trap 14:** PPO is **on-policy** (continuous actions)
 - 15 **Trap 15:** Double DQN reduces **overestimation bias**
- Remember:** RL is about **learning optimal behavior through experience!**

Reinforcement Learning - Complete Summary

Part 1 - Covered:

- 1 RL Fundamentals: Agent, Environment, States, Actions, Rewards
- 2 Multi-Armed Bandit: Exploration vs Exploitation, ϵ -Greedy
- 3 Monte Carlo Methods: Episode-based learning, returns
- 4 Temporal Difference: Bootstrapping, Q-learning, SARSA

Part 2 - Covered:

- 1 Deep RL & DQN: Neural networks, Experience replay, Target networks
- 2 Policy Methods: REINFORCE, Actor-Critic, Continuous actions
- 3 MDP & Bellman: Markov property, Dynamic Programming
- 4 Advanced Topics: RLHF, AlphaZero, SAC, PPO

Key Takeaway:

- RL is about [learning through experience](#)
- Balance [exploration and exploitation](#)
- Choose between [value-based](#) and [policy-based](#)
- Understand the [bias-variance trade-off](#)

Exam Success: Practice, understand concepts, and master the trade-offs!